

---

EEE5776. Applied Machine Learning



## **Module 2. Supervised Learning and Experimental Design**

**Decision Trees Classifier- Exercise**

**Decision Tree Regression**

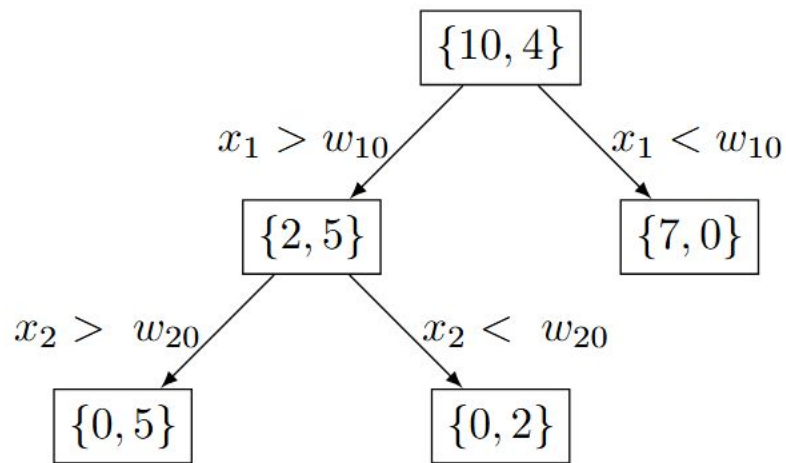
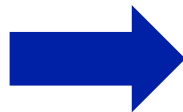
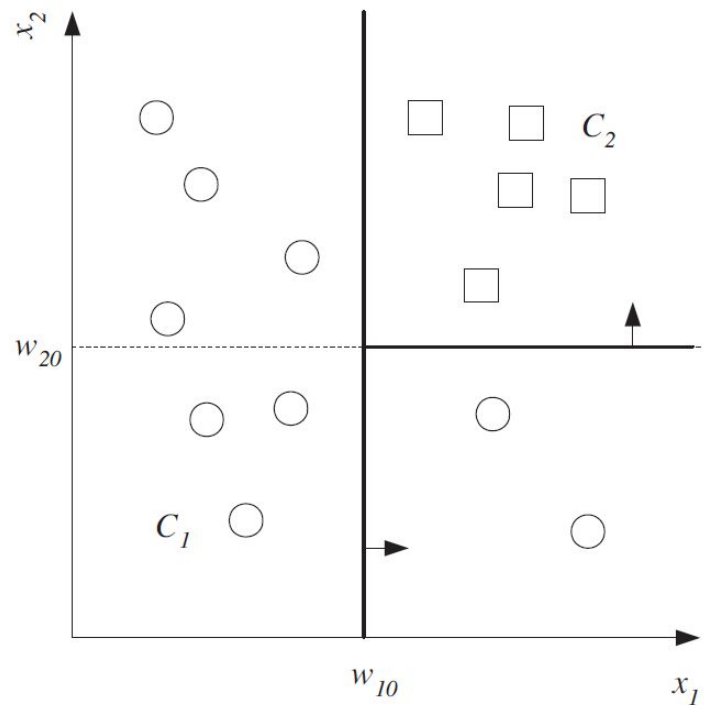
**Ensemble Learning - Random Forests**

**Model Comparison**



# Decision Tree Classifier - Exercise

# Intuition



# Impurity and Information Criteria

There are many models to measure the quality of the split. The most common impurity criteria are:

- **Entropy**: is an information gain split criterion. Measure of uncertainty or disorder in a dataset.

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

- **Gini Index**: is an impurity split criterion.

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

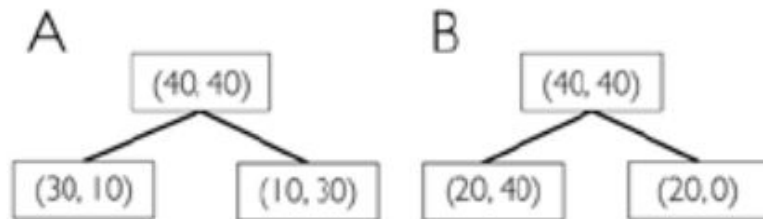
Where  $p_i$  is the proportion of samples that belong to class  $i$  for a particular node

**Information Gain**

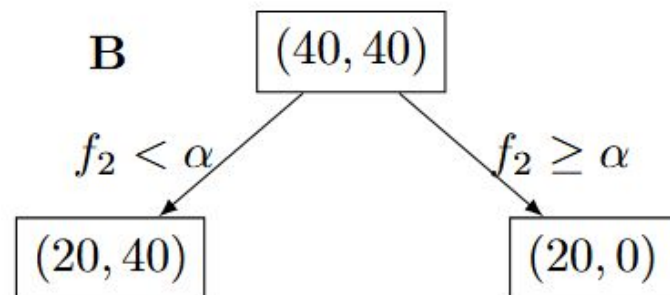
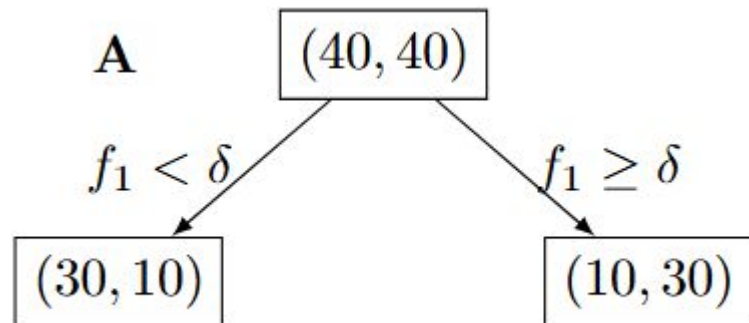
$$IG(D_p, f) = I(D_p) - \frac{N_{\text{left}}}{N_p} I(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I(D_{\text{right}})$$

# Exercise

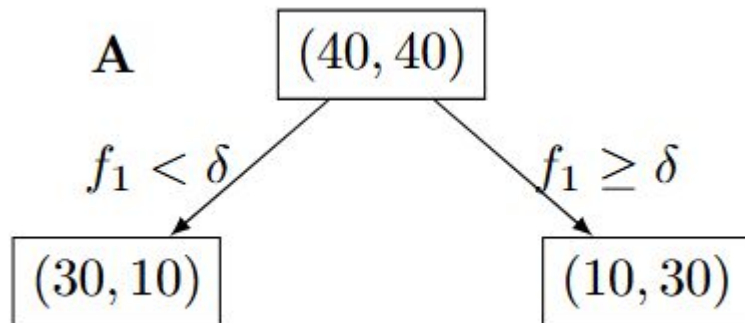
- Suppose that we have a dataset  $\{(\mathbf{x}_i, t_i)\}_{i=1}^N$ , where  $\mathbf{x}_i \in \mathbb{R}^2$  and  $t_i \in \{0, 1\}$
- We are dealing with a 2-dimensional feature space and 2 classes. Assume that we have 80 samples total in training, where 40 samples are from  $C_0$  and 40 samples from  $C_1$ . In constructing a tree, we must decide which feature (and at what threshold) should be used to split the parent node. The figure below illustrates the resulting tree when (A) using feature 1 to split the parent node, and (B) using feature 2 to split the parent node.
  - 1. Which decision tree would you choose?
  - 2. Compute the information gain using all impurity measures.



# Exercise



# Exercise



$$I(D_P) = 1 - \sum_{i=1}^c p_i^2 = 1 - \left(\frac{40}{40+40}\right)^2 - \left(\frac{40}{40+40}\right)^2 = \frac{1}{2}$$

$$I(D_{\text{LEFT}}) = 1 - \left(\frac{30}{30+10}\right)^2 - \left(\frac{10}{30+10}\right)^2 = \frac{3}{8}$$

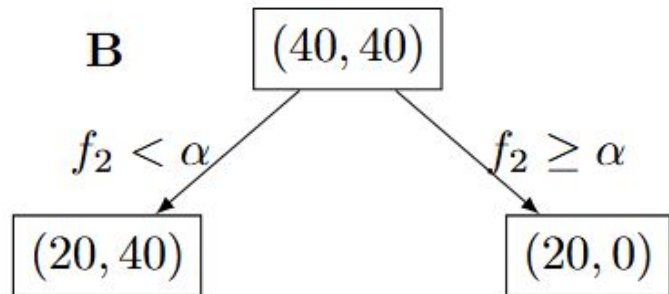
$$I(D_{\text{RIGHT}}) = 1 - \left(\frac{10}{10+30}\right)^2 - \left(\frac{30}{10+30}\right)^2 = \frac{3}{8}$$

$$\begin{aligned} IG(\text{TREE } A) &= \frac{1}{2} - \left(\frac{30+10}{40+40}\right) \frac{3}{8} - \left(\frac{10+30}{40+40}\right) \frac{3}{8} \\ &= 0.125 \end{aligned}$$

**Information Gain**

$$IG(D_p, f) = I(D_p) - \frac{N_{\text{left}}}{N_p} I(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I(D_{\text{right}})$$

# Exercise



$$I_G(\text{TREE}) = \dots \approx 0.16$$

Because ,  $I_G(B) > I_G(A)$  we pick B to split the parent node





# Decision Tree Regression

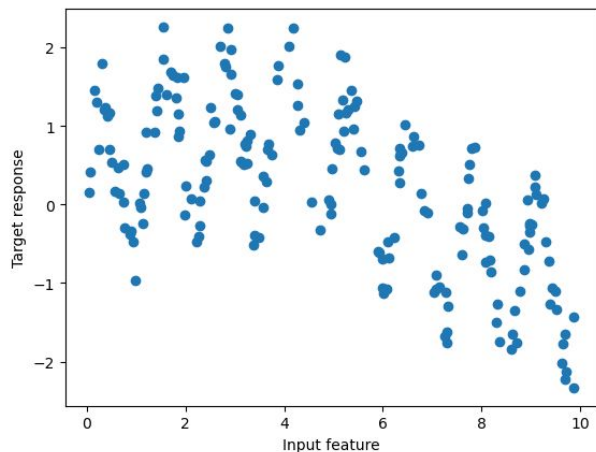
---

# Regression

- We've considered trees within the context of classification.
- Trees can also be made to work in the case of regression (**that is, continuous rather than categorical variables**).
- The estimator to use for this is the `DecisionTreeRegressor` or the `RandomForestRegressor`, and the syntax is very similar to what we saw earlier.

# Regression

Consider the following data,  
drawn from the combination of  
a fast and slow oscillation



Using the random forest  
regressor, we can find the best  
fit curve as follows:

```
from sklearn.tree import DecisionTreeRegressor
```

```
DecisionTreeRegressor?
```

```
dt_reg = DecisionTreeRegressor(max_depth=6)
dt_reg.fit(x[:, np.newaxis], t)
```

```
xfit = np.linspace(0, 10, 1000)
yfit = dt_reg.predict(xfit[:, np.newaxis])
ytrue = model(xfit, sigma=0)
```

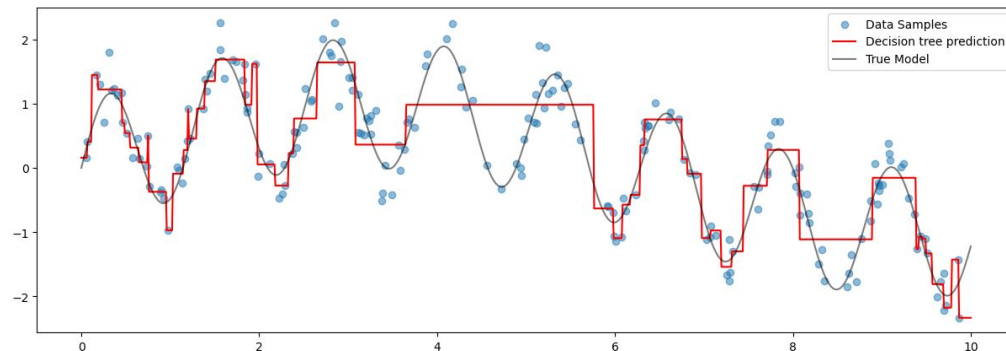
```
plt.figure(figsize=(15,5))
plt.scatter(x, t, alpha=0.5, label='Data Samples')
plt.plot(xfit, yfit, '-r', label='Decision tree prediction');
plt.plot(xfit, ytrue, '-k', alpha=0.5, label='True Model');
plt.legend();
```

# Regression

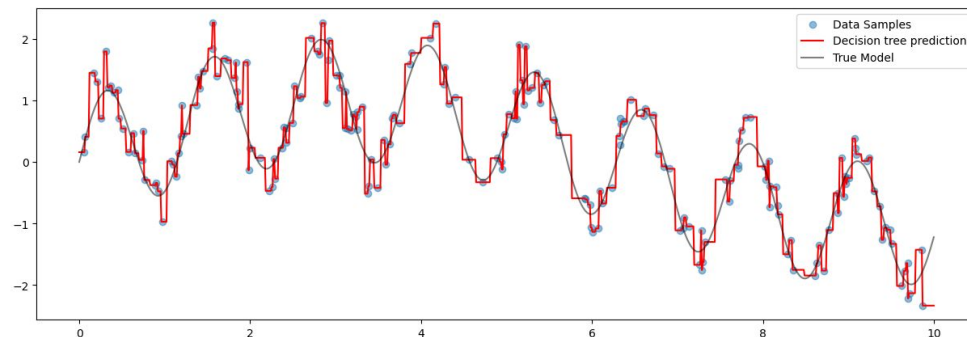
Here the true model is shown in the smooth gray curve, while the decision tree model is shown by the jagged red curve. As you can see, the non-parametric tree model is flexible enough to fit the multi-period data, without us needing to specifying a multi-period model!

The CART algorithm works mostly the same way as earlier, except that instead of trying to split the training set in a way that minimizes impurity, **it now tries to split the training set in a way that minimizes the MSE.**

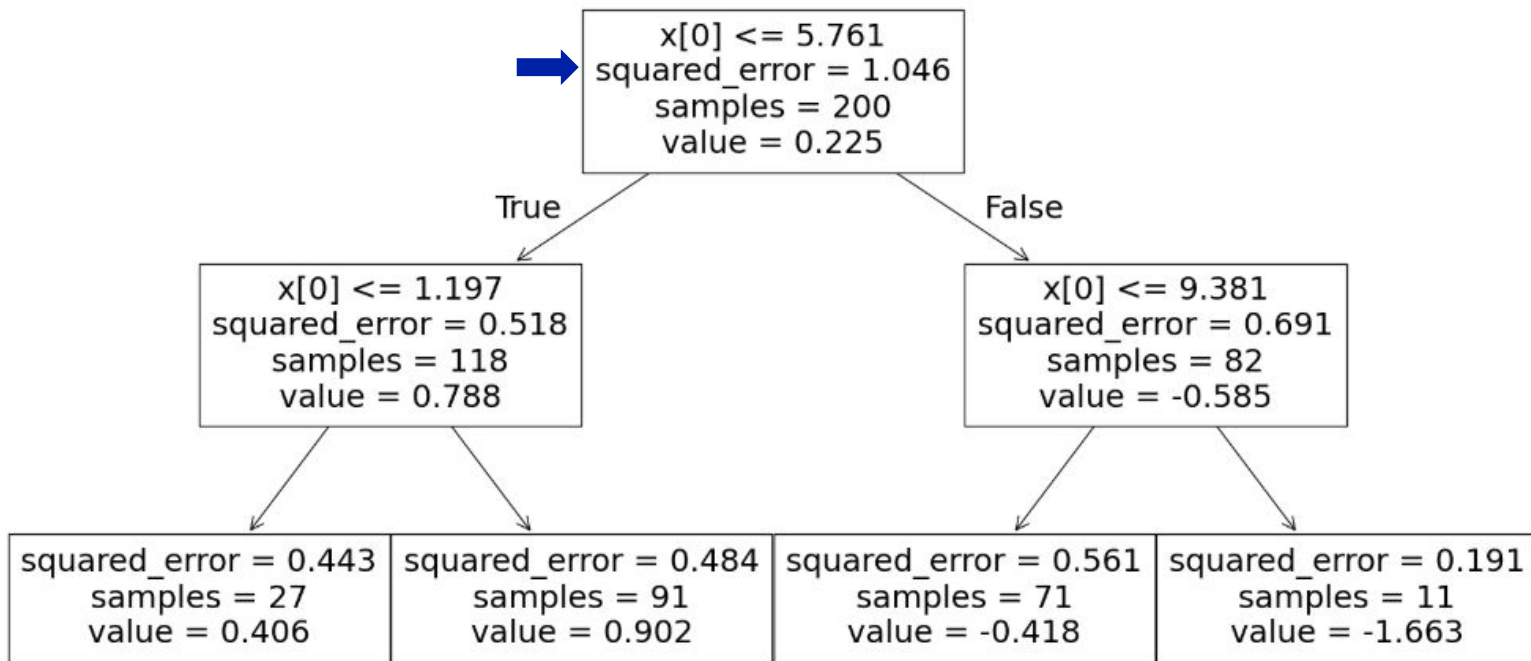
Depth=6



Depth=20



# Regression



A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of parallel paths that curve and then straighten out.

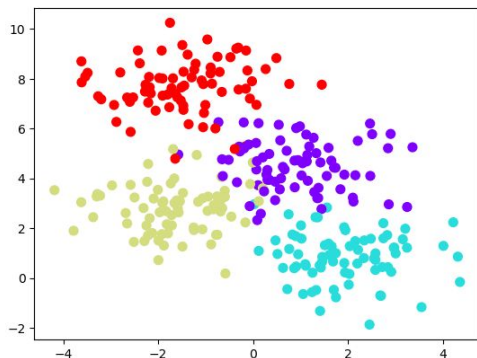
# Ensemble Learning - Random Forest

# Bagging

This notion - that multiple overfitting estimators can be combined to reduce the effect of this overfitting - is what underlies an ensemble method called **bagging**.

**Bagging**, which stands for bootstrap aggregating, makes use of an ensemble (a grab bag, perhaps) of parallel estimators, each of which overfits the data, and averages the results to find a better classification.

A bootstrap sample is a sample taken from the original data set with replacement. The bootstrap sample is the same size as the original. This type of bagging classification can be done manually using Scikit-Learn's BaggingClassifier meta-estimator, as shown here:



```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

BaggingClassifier?

tree = DecisionTreeClassifier()
bag = BaggingClassifier(tree, n_estimators=500,
                        bootstrap=True,
                        oob_score=True,
                        max_samples=0.8,
                        random_state=1)

bag.fit(X, t)
visualize_classifier(bag, X_train, t_train)
```

# Example

Dataset:



$n=8$

Bootstrap Sample 1:



$n=8$

Bootstrap Sample 2:

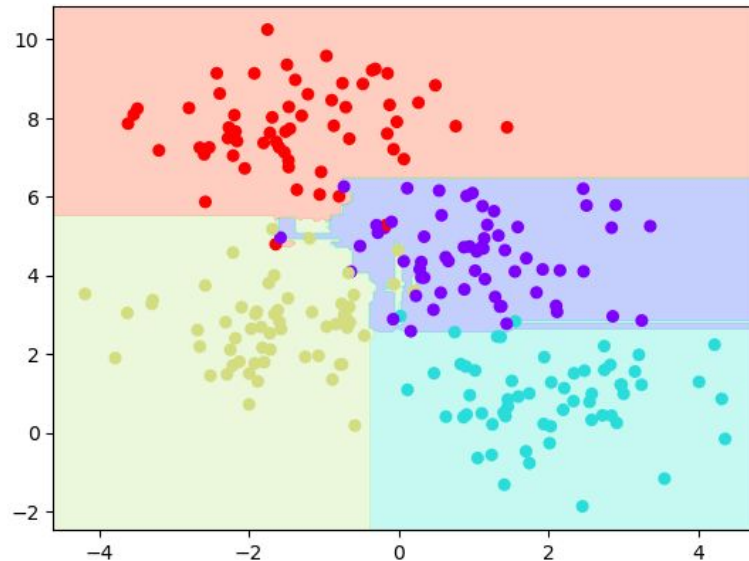


$n=8$



# Bagging

In this example, we have randomized the data by fitting each estimator with a random subset of 80% of the training points. In practice, decision trees are more effectively randomized by injecting some stochasticity in how the splits are chosen: this way all the data contributes to the fit each time, but the results of the fit still have the desired randomness.



# Out-of-Bag Evaluation

- With bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all.
- By default a BaggingClassifier samples  $m$  training instances with replacement (bootstrap=True), where  $m$  is the size of the training set.
- A bagging ensemble can be evaluated using oob instances, without the need for a separate validation set: indeed, if there are enough estimators, then each instance in the training set will likely be an oob instance of several estimators, so these estimators can be used to make a fair ensemble prediction for that instance. Once you have a prediction for each instance, you can compute the ensemble's prediction accuracy (or any other metric).

```
bag.oob_score_
```

```
0.9083333333333333
```

```
y_test = bag.predict(X_test)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(t_test, y_test)
```

```
0.9666666666666667
```

# Random Forests

An ensemble of randomized decision trees is known as a random forest.

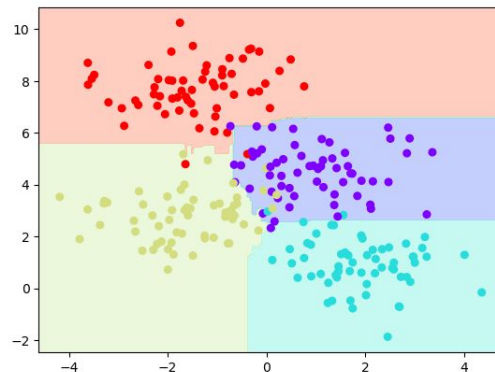
In Scikit-Learn, such an optimized ensemble of randomized decision trees is implemented in the `RandomForestClassifier` estimator, which takes care of all the randomization automatically. All you need to do is select a number of estimators, and it will very quickly (in parallel, if desired) fit the ensemble of trees:

```
from sklearn.ensemble import RandomForestClassifier

RandomForestClassifier?

model = RandomForestClassifier(n_estimators=100,
                              max_depth=4,
                              random_state=0)

visualize_classifier(model, X_train, t_train);
```



We see that by averaging over 100 randomly perturbed models, we end up with an overall model that is much closer to our intuition about how the parameter space should be split.

```
model.feature_importances_
array([0.35408312, 0.64591688])
```

# Hyperparameters

- With a few exceptions, a **RandomForestClassifier** has all the hyperparameters of a `DecisionTreeClassifier` (to control how trees are grown), plus all the hyperparameters of a `BaggingClassifier` to control the ensemble itself.
- The Random Forest algorithm introduces extra randomness when growing trees; instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features.
- The algorithm results in greater tree diversity, which (again) trades a higher bias for a lower variance generally yielding an overall better model.

# Summary

- Random forest is a type of ensemble estimators. Random forests are a powerful method with several **advantages**:
  - Both training and prediction are very fast, because of the simplicity of the underlying decision trees. In addition, both tasks can be straightforwardly parallelized, because the individual trees are entirely independent entities.
  - The multiple trees allow for a probabilistic classification: a majority vote among estimators gives an estimate of the probability (accessed in Scikit-Learn with the `predict_proba()` method).
  - The non-parametric model is extremely flexible, and can thus perform well on tasks that are underfit by other estimators.

A primary **disadvantage** of random forests is that the results are not easily interpretable: that is, if you would like to draw conclusions about the meaning of the classification model, random forests may not be the best choice.

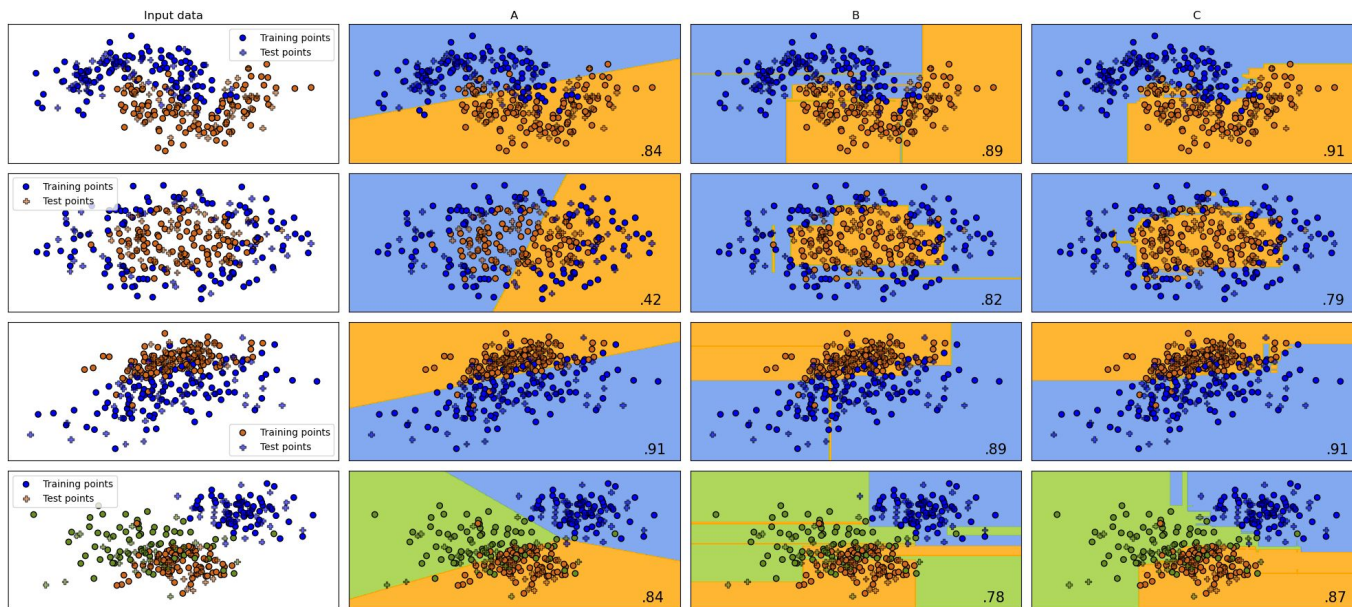
**The final decision is the majority of votes → This makes RF less explainable**

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps, extending from the top to the bottom.

# Model Comparison

---

# Comparing Logistic Regression, Decision Trees & Random Forests for Classification Tasks



Logistic  
Regression

Decision Trees

Random Forest